

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA0304

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2,BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks: 50

Annexure A

Case Study

Duration: 2hrs

1. A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays. Analyze how the components of a Markov Decision Process (states, actions, rewards, policy, and environment) can be defined for this system. Design a reward function that balances speed, accuracy, and safety, and evaluate how reward design influences the robot's learning and behavior. **(5 Marks)**
2. A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history. Apply RL concepts to construct a suitable state representation, action space, and reward function. Analyze how different reward signals affect user engagement and evaluate the trade-off between exploration and exploitation. **(5 Marks)**

3. An autonomous agricultural system uses RL to optimize irrigation and fertilizer usage based on soil conditions and weather forecasts. Design an RL framework by defining states, actions, and rewards. Analyze the effect of delayed rewards on crop yield optimization and justify the choice of a suitable learning algorithm.
(5 Marks)
4. A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making.
(5 Marks)
5. A ride-sharing company plans to use RL to optimize driver allocation and dynamic pricing strategies. Apply RL concepts to define states, actions, and rewards. Analyze how environmental uncertainty affects learning and evaluate ethical concerns related to pricing strategies.
(5 Marks)
6. A smart home automation system uses RL to control lighting, heating, and cooling based on occupancy and weather conditions. Design an RL framework including states, actions, and reward function. Analyze how conflicting objectives like comfort and energy saving can be handled and evaluate the impact of reward shaping.
(5 Marks)
7. A gaming company is developing an RL-based agent to play a complex strategy game against human players. Analyze how the problem can be modeled as an MDP. Design a reward structure that encourages long-term strategy and evaluate how exploration strategies affect performance.
(5 Marks)
8. A financial institution uses RL to detect fraudulent transactions in real time. Apply RL principles to define the state space, action space, and reward function. Analyze the challenges of imbalanced data and delayed feedback, and evaluate the risks associated with incorrect decisions.
(5 Marks)
9. A healthcare system uses RL to recommend personalized rehabilitation exercises for patients based on their progress. Design an RL framework with suitable states, actions, and rewards. Analyze the impact of delayed rewards and patient variability, and evaluate ethical concerns and safety considerations.
(5 Marks)

10. A smart grid system uses RL to manage electricity distribution and reduce peak load demand. Analyze how the components of an MDP can be defined in this context. Design a reward function to balance efficiency and reliability, and evaluate the challenges of scalability and real-time learning. **(5 Marks)**

Code of Conduct:

*I _Y .Thanushma (192425355) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CASESTUDY

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: RL-Based Warehouse Robot

Understanding of the Case

A logistics company is developing a warehouse robot using Reinforcement Learning (RL) to pick, pack, and transport items efficiently. The robot must minimize delivery time, avoid obstacles, and ensure safe and accurate handling of items. This problem is modeled as a Markov Decision Process (MDP), where the robot learns the best actions through interaction with the warehouse environment and rewards.

MDP Components

MDP Component	Definition for Warehouse Robot
States (S)	Robot's location, package status (picked/not picked), destination, battery level, obstacle positions, and shelf locations.
Actions (A)	Move forward, backward, left, right, pick an item, pack an item, deliver an item, recharge battery, or wait.
Environment (E)	Warehouse containing shelves, storage racks, workers, pathways, charging stations, obstacles, and delivery points.
Policy (π)	A strategy that selects the best action in each state to maximize long-term rewards while ensuring efficient and safe task completion.
Rewards (R)	Positive rewards for successful deliveries and negative rewards for delays, collisions, wrong picks, and unnecessary movements.

Reward Function Design

Action	Reward
Successful delivery	+100
Correct item picked	+20
Fast task completion	+10
Collision with obstacle	-20

Wrong item picked	-10
Extra movement	-2
Delay in delivery	-5

Analysis

The reward function encourages the robot to complete tasks quickly, accurately, and safely. Positive rewards motivate successful deliveries and correct item selection, while penalties reduce delays, unnecessary movements, and collisions. This helps the robot learn the shortest and safest routes, improving warehouse efficiency.

Evaluation

A balanced reward function improves the robot's learning and behavior by maintaining speed, accuracy, and safety. If speed is rewarded too much, the robot may ignore safety. If penalties are too high, it may become overly cautious. Therefore, a balanced reward function helps the robot learn an optimal policy for reliable warehouse operations.

Conclusion

By modeling the warehouse robot as a Markov Decision Process (MDP) with well-defined states, actions, environment, policy, and reward function, the RL agent learns efficient warehouse operations. This results in faster deliveries, accurate package handling, safe navigation, and improved overall productivity.

Case Study 2: RL-Based Movie Recommendation System

Understanding of the Case

A streaming platform aims to use Reinforcement Learning (RL) to recommend movies dynamically based on a user's preferences and viewing history. The system learns from user interactions and continuously improves its recommendations to maximize user satisfaction and engagement.

RL Components

RL Component	Definition for Movie Recommendation System
States (S)	User's viewing history, watch time, liked/disliked movies, preferred genres, ratings, and time of day.
Actions (A)	Recommend a movie or TV show from different genres or categories.
Environment (E)	Streaming platform containing users, movies, TV shows, ratings, and user interactions.
Policy (π)	A strategy that recommends the best movie based on the user's current preferences and previous interactions.
Rewards (R)	Positive rewards for user engagement and negative rewards for skipped, disliked, or abandoned recommendations.

Reward Function Design

Action	Reward
User watches recommended movie completely	+100
User likes or rates the movie positively	+20
User adds movie to watchlist	+10
User skips the recommendation	-10
User stops watching midway	-20
User dislikes the movie	-30

Analysis

The reward function encourages the system to recommend movies that users are likely to watch and enjoy. Positive rewards increase user engagement by promoting relevant recommendations, while negative rewards reduce repeated recommendations of unwanted content. This helps the system learn user preferences over time.

Evaluation

The system must balance exploration and exploitation. Exploration recommends new or less-known movies to discover changing user interests, while exploitation recommends movies similar to those the user already enjoys. Too much exploration may reduce satisfaction, whereas too much exploitation may make recommendations repetitive. A balanced approach improves long-term user engagement and recommendation quality.

Conclusion

By applying Reinforcement Learning, the streaming platform can continuously improve its movie recommendations based on user feedback. A well-designed state representation, action space, and balanced reward function help maximize user engagement while effectively balancing exploration and exploitation for personalized recommendations.

Case Study 3: RL-Based Smart Agricultural System

Understanding of the Case

A smart agricultural system uses Reinforcement Learning (RL) to automatically manage irrigation and fertilizer usage based on soil conditions and weather forecasts. The objective is to maximize crop yield while minimizing water and fertilizer consumption. The RL agent learns the best farming decisions through continuous interaction with the environment.

State, Action and Reward Design

States (S):

- Soil moisture level
- Soil nutrient level
- Temperature and humidity
- Weather forecast
- Crop growth stage
- Water availability

Actions (A):

- Increase irrigation
- Reduce irrigation
- Apply fertilizer
- Adjust fertilizer quantity
- Delay irrigation
- No action

Reward Function (R):

Condition	Reward
Healthy crop growth	+50
High crop yield	+100
Efficient water usage	+20
Efficient fertilizer usage	+20
Water wastage	-20
Excess fertilizer usage	-15
Poor crop growth	-30

Analysis

The agricultural system receives delayed rewards because the effects of irrigation and fertilizer are observed only after the crops grow. Instead of rewarding immediate actions, the agent learns which decisions produce better crop yield over time. This helps the system optimize long-term farming performance rather than short-term gains.

Evaluation

Q-Learning is a suitable learning algorithm because it learns the best actions through trial and error without requiring prior knowledge of the environment. It enables the system to continuously improve irrigation and fertilizer decisions under changing soil and weather conditions, resulting in higher productivity and efficient resource utilization.

Conclusion

A Reinforcement Learning-based agricultural system can intelligently manage irrigation and fertilizer by learning from environmental feedback. Considering delayed rewards and using Q-Learning helps achieve sustainable farming with improved crop yield, reduced water consumption, and efficient fertilizer usage.

Case Study 4: RL-Based Cybersecurity Intrusion Detection System

Understanding of the Case

A cybersecurity system uses Reinforcement Learning (RL) to detect and respond to network attacks in real time. The RL agent continuously monitors network traffic, identifies suspicious activities, and selects appropriate actions to protect the system from cyber threats while minimizing false alarms.

State, Environment, Actions and Reward Design

States (S):

- Network traffic status
- Number of failed login attempts
- Unusual IP addresses
- Traffic volume
- Suspicious user behavior
- System resource usage

Environment (E):

- Computer network containing users, servers, firewalls, routers, network traffic, and possible cyber attacks.

Actions (A):

- Allow network traffic
- Block suspicious IP address
- Generate security alert

- Isolate infected device
- Request user authentication
- Continue monitoring

Reward Function

Action	Reward
Correctly detects an attack	+100
Successfully blocks malicious traffic	+50
Prevents system damage	+30
False alarm	-20
Missed attack	-100
Delayed response	-30

Analysis

In cybersecurity, rewards are often sparse because attacks occur infrequently. The RL agent may receive rewards only when an attack is detected or prevented, making learning slower. The agent must learn to recognize attack patterns even when immediate rewards are unavailable.

Evaluation

Real-time decision-making is essential because cyber attacks spread rapidly. The RL agent must detect threats quickly and respond immediately without affecting normal network traffic. A balanced reward function helps reduce false positives while ensuring accurate and timely intrusion detection.

Conclusion

An RL-based cybersecurity system improves network security by learning from real-time interactions. By modeling appropriate states, environment, actions, and reward functions, the agent can accurately detect and respond to attacks. Handling sparse rewards and making fast decisions enables reliable intrusion detection and stronger cyber defense.

Case Study 5: RL-Based Ride-Sharing System

Understanding of the Case

A ride-sharing company plans to use Reinforcement Learning (RL) to optimize driver allocation and dynamic pricing strategies. The RL agent learns from customer demand, driver availability, and traffic conditions to improve ride efficiency while maintaining fair pricing.

RL Components

RL Component	Definition for Ride-Sharing System
States (S)	Customer demand, driver availability, traffic conditions, weather, current fare, and pickup locations.
Actions (A)	Assign driver, reassign driver, increase fare, decrease fare, or keep fare unchanged.
Environment (E)	Road network containing customers, drivers, traffic conditions, weather, and ride requests.
Policy (π)	A strategy that selects the best driver allocation and pricing decision to maximize service efficiency and customer satisfaction.
Rewards (R)	Positive rewards for successful rides, reduced waiting time, and efficient driver allocation, and negative rewards for cancellations, delays, or unfair pricing.

Reward Function Design

Action	Reward
Successful ride completion	+100
Reduced customer waiting time	+30
Efficient driver allocation	+20
Ride cancellation	-20

Long waiting time	-15
Unfair pricing	-25

Analysis

The ride-sharing environment is highly dynamic because traffic conditions, customer demand, and driver availability change continuously. The RL agent must adapt its decisions to these uncertainties for efficient ride allocation.

Evaluation

Dynamic pricing should be fair and transparent. Charging very high prices during emergencies or peak hours may reduce customer trust. Therefore, the RL system should balance profitability with fairness and customer satisfaction.

Conclusion

By applying Reinforcement Learning, the ride-sharing system can optimize driver allocation and pricing decisions. Considering environmental uncertainty and ethical pricing improves service quality, customer satisfaction, and overall operational efficiency.

Case Study 6: RL-Based Smart Home Automation System

Understanding of the Case

A smart home automation system uses **Reinforcement Learning (RL)** to control lighting, heating, and cooling based on occupancy and weather conditions. The objective is to improve user comfort while minimizing energy consumption.

RL Framework

States (S):

- Room occupancy
- Indoor temperature
- Outdoor weather
- Time of day

- Light intensity
- Energy consumption

Actions (A):

- Turn lights ON/OFF
- Increase or decrease heating
- Increase or decrease cooling
- Adjust thermostat
- Maintain current settings

Reward Function

Condition	Reward
User comfort maintained	+50
Energy saved	+40
Efficient temperature control	+20
Excessive energy usage	−20
User discomfort	−30
Unnecessary appliance usage	−10

Analysis

The system has conflicting objectives because increasing comfort often increases energy consumption. The RL agent learns to balance these objectives by selecting actions that provide acceptable comfort while reducing unnecessary energy usage.

Evaluation

Reward shaping improves learning by providing intermediate rewards for energy-efficient actions instead of rewarding only the final outcome. This enables the agent to learn faster and achieve a better balance between comfort and energy efficiency.

Conclusion

An RL-based smart home automation system intelligently manages lighting, heating, and cooling. By using a well-designed reward function and reward shaping, it provides comfortable living conditions while reducing energy consumption and operating costs.

Case Study 7: RL-Based Strategy Game Agent

Understanding of the Case

A gaming company is developing a Reinforcement Learning (RL) agent to play a complex strategy game against human players. The agent learns by interacting with the game environment and aims to maximize its chances of winning by making strategic decisions over multiple moves. The problem can be modeled as a Markov Decision Process (MDP).

MDP Components

MDP Component	Definition for Strategy Game
States (S)	Current game board, player position, available resources, opponent's moves, score, and remaining time.
Actions (A)	Attack, defend, collect resources, move, build structures, or use special abilities.
Environment (E)	Game world containing players, opponents, game rules, resources, and obstacles.
Policy (π)	A strategy that selects the best move in each game state to maximize the chances of winning.
Rewards (R)	Positive rewards for winning battles, collecting resources, and winning the game, and negative rewards for losing resources or making poor moves.

Reward Function Design

Action	Reward
Win the game	+100
Defeat an opponent	+50
Collect resources	+20
Complete mission	+30
Lose resources	-20
Lose the game	-100

Analysis

The reward structure encourages long-term strategy by giving higher rewards for achieving the final objective rather than only immediate gains. The RL agent learns to plan future moves instead of focusing only on short-term rewards.

Evaluation

Exploration strategies help the agent discover new strategies and improve gameplay. Excessive exploration may reduce short-term performance, while too much exploitation may prevent learning better strategies. A balanced approach improves overall game performance.

Conclusion

By modeling the game as a Markov Decision Process (MDP) with appropriate states, actions, environment, policy, and reward function, the RL agent can develop effective long-term strategies and achieve better performance against human players.

Case Study 8: RL-Based Financial Fraud Detection System

Understanding of the Case

A financial institution uses Reinforcement Learning (RL) to detect fraudulent transactions in real time. The system learns from transaction patterns and customer behavior to identify suspicious activities while reducing financial losses and minimizing false alarms.

State, Action and Reward Design

States (S):

- Transaction amount
- Customer transaction history
- Transaction location
- Time of transaction
- Device/IP address
- Risk score

Actions (A):

- Approve transaction
- Block transaction
- Request additional verification
- Send for manual review

Reward Function

Condition	Reward
Correct fraud detection	+100
Correct approval of genuine transaction	+50
Prevent financial loss	+30
False alarm	-20
Missed fraudulent transaction	-100
Delayed response	-30

Analysis

The fraud detection system faces imbalanced data, as genuine transactions are much more common than fraudulent ones. It also experiences delayed feedback because the fraud status of

some transactions is confirmed only after investigation. These factors make the learning process more challenging.

Evaluation

Incorrect decisions can have serious consequences. Blocking genuine transactions affects customer satisfaction, while approving fraudulent transactions leads to financial losses and security risks. Therefore, the RL agent must balance security with smooth customer experience.

Conclusion

An RL-based fraud detection system can improve real-time transaction monitoring by learning from transaction patterns. A well-designed state space, action space, and reward function help detect fraud accurately while reducing false alarms and protecting customers and financial institutions.

Case Study 9: RL-Based Personalized Rehabilitation System

Understanding of the Case

A healthcare system uses Reinforcement Learning (RL) to recommend personalized rehabilitation exercises based on patient recovery progress. The RL agent learns from patient responses and continuously updates exercise plans to improve recovery while ensuring patient safety.

RL Components

RL Component	Definition for Rehabilitation System
States (S)	Patient's recovery level, pain level, muscle strength, range of motion, previous exercise performance, and health condition.
Actions (A)	Recommend light, moderate, or advanced exercises, increase or decrease exercise intensity, or provide a rest period.
Environment (E)	Healthcare environment containing patients, rehabilitation equipment, therapists, and medical records.

Policy (π)	A strategy that selects the most suitable exercise plan based on the patient's current recovery status.
Rewards (R)	Positive rewards for improved recovery and successful exercise completion, and negative rewards for pain, injury, or missed rehabilitation sessions.

Reward Function Design

Action	Reward
Improvement in recovery	+100
Successful exercise completion	+40
Improved mobility	+30
Patient discomfort	-20
Injury or excessive pain	-100
Missed rehabilitation session	-10

Analysis

The rehabilitation system receives delayed rewards because patient improvement becomes visible only after several therapy sessions. Patient variability, such as age, health condition, and recovery speed, requires the RL agent to personalize exercise recommendations for each individual.

Evaluation

Ethical concerns include patient privacy, fairness, and safety. Incorrect exercise recommendations may lead to injury, so the RL system should operate under medical supervision and ensure safe treatment decisions.

Conclusion

By applying Reinforcement Learning, the healthcare system can provide personalized rehabilitation programs. A well-designed reward function and careful consideration of delayed rewards, patient variability, and safety improve recovery outcomes and treatment quality.

Case Study 10: RL-Based Smart Grid System

Understanding of the Case

A smart grid system uses Reinforcement Learning (RL) to manage electricity distribution and reduce peak load demand. The system is modeled as a Markov Decision Process (MDP), where the RL agent learns the best power distribution strategy by interacting with the electrical grid.

MDP Components

MDP Component	Definition for Smart Grid System
States (S)	Electricity demand, power generation, battery storage level, grid load, renewable energy availability, and weather conditions.
Actions (A)	Increase or decrease power supply, store excess energy, release stored energy, or balance electricity load.
Environment (E)	Smart grid containing consumers, power plants, renewable energy sources, storage systems, and transmission lines.
Policy (π)	A strategy that selects the best power distribution action to maintain efficiency and grid reliability.
Rewards (R)	Positive rewards for efficient power distribution and stable grid operation, and negative rewards for energy wastage, overloads, and power outages.

Reward Function Design

Action	Reward
Efficient power distribution	+100
Reduced peak load	+50
Stable electricity supply	+30
Energy wastage	-20
Grid overload	-50

Power outage	-100
--------------	------

Analysis

The reward function balances efficiency by reducing energy wastage and reliability by maintaining a stable electricity supply. The RL agent learns to distribute power effectively under changing demand conditions.

Evaluation

The smart grid faces challenges in scalability due to a large number of users and power sources. It also requires real-time learning to quickly respond to changes in electricity demand and maintain uninterrupted power supply.

Conclusion

By modeling the smart grid as a Markov Decision Process (MDP) with well-defined states, actions, environment, policy, and reward function, the RL agent can improve electricity distribution, reduce peak load, enhance efficiency, and ensure reliable power delivery.